SLOT 8

Named Routes & Arguments, Navigator 2.0 Concepts, Android Deep Linking v

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Named Routes & Arguments
- Navigator 2.0 Concepts
- Android Deep Linking

Mục tiêu tổng quát của Slot 7

- Hiểu rõ **3 cấp độ điều hướng**:
  1. Điều hướng cơ bản (Navigator 1.0)
  2. Điều hướng có cấu trúc (Named Routes)
  3. Điều hướng theo trạng thái & hệ thống (Navigator 2.0 + Deep Link)
- Có tư duy **thiết kế luồng người dùng** thay vì chỉ viết code điều hướng
- Sẵn sàng học:
  - GoRouter / AutoRoute
  - App nhiều role – nhiều entry point
  - Navigation trong dự án lớn

1 NAMED ROUTES & ARGUMENTS

Khái niệm cốt lõi cần nắm

- **Named Route**: điều hướng bằng *tên route* (`'/login'`, `'/detail'`) thay vì gọi trực tiếp widget.
- **Arguments**: truyền dữ liệu giữa các màn hình thông qua route.
- Tách **logic điều hướng** khỏi **UI cụ thể**.

Mục tiêu học tập

Sau nội dung này, sinh viên có thể:

- Khai báo và sử dụng `routes`, `onGenerateRoute`
- Điều hướng giữa nhiều màn hình bằng tên route
- Truyền & nhận dữ liệu (arguments) an toàn, rõ ràng
- Quản lý luồng app đơn giản (list → detail, login → home)

Mô tả sư phạm (vì sao cần học?)

- Giải quyết vấn đề **app nhiều màn hình trở nên rối** khi dùng `Navigator.push()` trực tiếp.
- Là nền tảng bắt buộc trước khi tiếp cận **Navigator 2.0** và **Deep Linking**.
- Giúp sinh viên **hình dung app như một sơ đồ màn hình**, không chỉ là widget rời rạc.

2 NAVIGATOR 2.0 – CONCEPTS & TƯ DUY NỀN TẢNG

Khái niệm cốt lõi cần nắm

- **State-based Navigation**: điều hướng dựa trên *trạng thái ứng dụng*, không chỉ hành động click.
- **Page stack** được mô tả bằng **data/state**, không thao tác trực tiếp push/pop.
- Phân biệt tư duy:
  - Navigator 1.0 → *imperative* (làm gì thì push đó)
  - Navigator 2.0 → *declarative* (state thế nào thì UI & route như vậy)

Mục tiêu học tập

Sau nội dung này, sinh viên có thể:

- Hiểu **vì sao Navigator 2.0 ra đời**
- Đọc – hiểu kiến trúc điều hướng mới (Router, Pages, Back button)
- Nhận biết **khi nào cần Navigator 2.0, khi nào KHÔNG cần**

PhucPT10@fpt.edu.vn

- Chuẩn bị tư duy để dùng các framework cao hơn (GoRouter, AutoRoute)

📌 Mô tả sư phạm

- Giải quyết bài toán **app phức tạp**: đăng nhập, phân quyền, deep link, refresh web.
- Giúp sinh viên **thoát khỏi tư duy "click → push"**, chuyển sang tư duy **user flow & app state**.
- Không tập trung code nặng, mà tập trung **hiểu bản chất điều hướng hiện đại**.

3 ANDROID DEEP LINKING TRONG FLUTTER

◈ Khái niệm cốt lõi cần nắm

- **Deep Link**: mở *trực tiếp một màn hình cụ thể* trong app bằng URL.
- Phân biệt:
  - App mở từ icon
  - App mở từ link (email, QR, web, notification)
- Flutter cần phối hợp với **Android system (Intent, manifest)**.

🎯 Mục tiêu học tập

Sau nội dung này, sinh viên có thể:

- Hiểu cách app Flutter nhận link từ bên ngoài
- Liên hệ deep link với **Named Routes / Navigator 2.0**
- Phân tích luồng user khi:
  - App đang mở
  - App đang chạy nền
  - App chưa từng mở

📌 Mô tả sư phạm

- Giải quyết nhu cầu **app thật ngoài đời**: marketing link, chia sẻ nội dung, thông báo.
- Giúp sinh viên hiểu rằng **điều hướng không chỉ xảy ra trong app**, mà còn đến từ hệ điều hành.
- Là cầu nối giữa Flutter và **hệ sinh thái Android thực tế**.

🗄 NGUỒN TÀI LIỆU THAM KHẢO (ƯU TIÊN DỄ TIẾP CẬN)

1. **Flutter Official – Navigation & Routing**
   👉 https://docs.flutter.dev/ui/navigation
2. **Flutter Cookbook – Navigation**
   👉 Các ví dụ ngắn, thực tế, dễ học theo
3. **Navigator 2.0 Overview (Flutter Team)**
   👉 https://docs.flutter.dev/ui/navigation/router
4. **Android Developers – Deep Links**
   👉 https://developer.android.com/training/app-links
5. **YouTube – Flutter Team / Reso Coder / Code With Andrea**
   👉 Video giải thích chậm, có ví dụ user flow rõ ràng

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1 NAMED ROUTES & PASSING ARGUMENTS

◈ Khái niệm quan trọng (Core Concept)

- **Named Route** là cách **đặt tên cho màn hình**, không phải gọi trực tiếp widget.
- Điều hướng dựa trên **route name** giúp app có cấu trúc rõ ràng hơn.
- **Arguments** là dữ liệu gửi kèm khi điều hướng (ví dụ: id, object).
- ❗ **Dễ hiểu sai nhất**:
  - Nghĩ rằng Named Route chỉ là "viết cho đẹp hơn"
  - Không phân biệt **route name** và **Widget class**

◈ Kỹ thuật / thao tác PHẢI THÀNH THẠO

Sinh viên **bắt buộc làm được**:

- Khai báo `routes` hoặc `onGenerateRoute`
- Gọi `Navigator.pushNamed()`
- Truyền dữ liệu qua `arguments`
- Nhận dữ liệu ở màn hình đích một cách an toàn
- Điều hướng đúng luồng:
  - List → Detail
  - Login → Home → Profile

◈ Lỗi thường gặp của sinh viên mới

- **Lỗi**: App chạy nhưng màn hình nhận `null`
  - 👉 Nguyên nhân: quên truyền hoặc ép kiểu sai `arguments`
- **Lỗi**: Điều hướng nhầm màn hình
  - 👉 Nguyên nhân: hard-code route name, không quản lý tập trung
- **Lỗi**: Code điều hướng rải rác, khó debug
  - 👉 Nguyên nhân: chưa hiểu vai trò của Named Routes trong tổ chức app

◈ Best Practices (nên áp dụng sớm)

- Định nghĩa **route name dưới dạng hằng số**
- Quản lý routes **tập trung tại 1 nơi**
- Chỉ truyền **dữ liệu cần thiết**, tránh truyền object quá lớn
- Luôn nghĩ: *"Màn hình này được mở từ đâu?"*

📘 Mô tả sư phạm

Named Routes giúp sinh viên **nhìn app như một sơ đồ màn hình**, không phải các widget rời rạc. Đây là bước bắt buộc trước khi làm app có nhiều luồng và nhiều điểm vào.

2 NAVIGATOR 2.0 – CONCEPTS & DECLARATIVE NAVIGATION

◈ Khái niệm quan trọng (Core Concept)

- **Navigator 2.0** không tập trung vào *push/pop*, mà tập trung vào **trạng thái app**.
- **Declarative Navigation**:
  👉 *State thế nào → màn hình hiển thị như vậy*
- Page stack được **mô tả bằng dữ liệu**, không điều khiển trực tiếp.
- ❗ **Dễ hiểu sai nhất**:
  - Nghĩ Navigator 2.0 là "Navigator 1.0 viết khác đi"
  - Cố gắng code Navigator 2.0 khi chưa cần

◈ Kỹ thuật / thao tác PHẢI NẮM (mức nhận thức)

Sinh viên **chưa cần code sâu**, nhưng **phải hiểu**:

- Vì sao Flutter cần Navigator 2.0

PhucPT10@fpt.edu.vn

- Sự khác nhau giữa:
  - Action-based navigation (bấm nút → push)
  - State-based navigation (login xong → tự sang home)
- Vai trò của:
  - Router
  - Page list
  - App state

◈ Lỗi thường gặp của sinh viên mới

- **Lỗi**: Áp dụng Navigator 2.0 cho app nhỏ
  - 👉 Nguyên nhân: hiểu sai mục đích của Navigator 2.0
- **Lỗi**: Không hiểu vì sao back button hoạt động sai
  - 👉 Nguyên nhân: chưa hiểu page stack được tạo từ state
- **Lỗi**: Học thuộc API nhưng không hiểu luồng
  - 👉 Nguyên nhân: học vẹt, thiếu liên hệ user flow

◈ Best Practices

- Chỉ dùng Navigator 2.0 khi:
  - App có nhiều trạng thái (login, role, deep link)
- Tập trung **hiểu tư duy**, không ép code
- Xem Navigator 2.0 như **nền tảng cho GoRouter / AutoRoute**

📄 Mô tả sư phạm

Navigator 2.0 giúp sinh viên **thoát khỏi tư duy "bấm thì push"**,
và bắt đầu suy nghĩ như người thiết kế **luồng người dùng trong app thật**.

3 ANDROID DEEP LINKING TRONG FLUTTER

◈ Khái niệm quan trọng (Core Concept)

- **Deep Link** là link mở **thẳng vào một màn hình cụ thể trong app**.
- App Flutter có thể được mở:
  - Từ icon
  - Từ notification
  - Từ link ngoài (email, web, QR)
- ❗ **Dễ hiểu sai nhất**:
  - Nghĩ deep link chỉ là vấn đề Flutter
  - Không phân biệt app đang mở hay chưa mở

◈ Kỹ thuật / thao tác PHẢI NẮM

Sinh viên cần:

- Hiểu luồng hoạt động của deep link trên Android
- Biết deep link **ảnh hưởng trực tiếp đến điều hướng**
- Liên hệ deep link với:
  - Named Routes
  - Navigator 2.0
- Phân tích được các tình huống:
  - App đã chạy
  - App đang nền
  - App chưa mở

◈ Lỗi thường gặp

- **Lỗi**: Deep link mở app nhưng vào sai màn hình

PhucPT10@fpt.edu.vn

    - 👉 Nguyên nhân: không map link với route đúng
- **Lỗi**: Crash khi mở link
    - 👉 Nguyên nhân: thiếu dữ liệu bắt buộc cho màn hình
- **Lỗi**: Không test đủ các trạng thái app
    - 👉 Nguyên nhân: tư duy test chỉ trong app

◇ Best Practices

- Xem deep link là **một điểm vào app (entry point)** ngang với icon
- Luôn chuẩn bị **fallback screen** khi thiếu dữ liệu
- Thiết kế route sao cho:
    - Có thể mở trực tiếp
    - Không phụ thuộc luồng trước đó

📘 Mô tả sư phạm

Deep Linking buộc sinh viên phải nghĩ rằng:
App không chỉ bắt đầu từ màn hình đầu tiên,
mà có thể bắt đầu từ bất kỳ màn hình nào.

✅ TỔNG KẾT PHÂN LOẠI MỨC ĐỘ

🟢 CỐT LÕI – BẮT BUỘC PHẢI NẮM

- Named Routes
- Passing Arguments
- Tư duy state-based navigation
- Deep link ảnh hưởng đến điều hướng

⚪ NÂNG CAO – NHẬN DIỆN (CHƯA CẦN CODE)

- Navigator 2.0 API chi tiết
- RouterDelegate / RouteInformationParser
- Web URL synchronization

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

1 VAI TRÒ CỦA SLOT 8 TRONG DỰ ÁN FLUTTER THỰC TẾ

Thực tế khi làm app

- App **không còn là demo 2–3 màn hình**
- App thật luôn có:
  - nhiều màn hình
  - nhiều luồng người dùng
  - nhiều cách mở app (icon, link, notification)

👉 **Navigation lúc này trở thành "xương sống" của app**

Các kỹ thuật Slot 8 được dùng như thế nào?

Xây dựng luồng điều hướng rõ ràng

- **Named Routes**:
  - đặt tên cho từng màn hình
  - nhìn app như *bản đồ màn hình*
- Giúp:
  - dễ đọc
  - dễ sửa
  - dễ giải thích cho team

Quản lý trạng thái & lịch sử điều hướng

- Navigator 1.0: phù hợp cho **luồng đơn giản**
- Navigator 2.0 (concept):
  - hiểu rằng **màn hình phụ thuộc vào trạng thái**
  - ví dụ:
    - chưa login → chỉ thấy Login
    - login xong → tự sang Home

👉 Không phải "bấm thì push", mà là **state đổi → màn hình đổi**

Mở app từ bên ngoài

- **Android Deep Linking**:
  - mở app từ:
    - link chia sẻ
    - notification
    - email
- Navigation phải:
  - mở **đúng màn hình**
  - có thể **bỏ qua các màn hình trước đó**

Chuẩn bị cho app mở rộng về sau

- Slot 8 giúp:
  - không phải *đập đi làm lại navigation*
  - sẵn sàng dùng GoRouter / AutoRoute sau này
- 👉 Đây là bước **chuyển từ app demo → app thật**

2 CÁC TÌNH HUỐNG THỰC TẾ (PROJECT-LEVEL SCENARIOS)

TÌNH HUỐNG 1: APP SINH VIÊN – LIST → DETAIL

Bối cảnh

- App hiển thị:
  - danh sách sinh viên

PhucPT10@fpt.edu.vn

    - click → xem chi tiết sinh viên

❗ Vấn đề gặp phải

- Dùng `Navigator.push()` trực tiếp:
    - code điều hướng rải rác
    - khó biết màn hình Detail được mở từ đâu
- Truyền dữ liệu:
    - lúc có, lúc null
    - sinh viên không hiểu vì sao lỗi

🛠 Cách áp dụng Slot 8

- Dùng **Named Routes**:
    - `/students`
    - `/student-detail`
- Truyền **arguments**:
    - id sinh viên hoặc object đơn giản
- Điều hướng:
    - list → detail theo route name

🎯 Mục tiêu đạt được

- Luồng rõ ràng
- Dễ debug
- Dễ mở rộng:
    - thêm "edit"
    - thêm "favorite"
- Sinh viên **bắt đầu tư duy app theo sơ đồ màn hình**

◈ TÌNH HUỐNG 2: APP BÁN HÀNG – HOME → PRODUCT → CART

🔑 Bối cảnh

- App ecommerce nhỏ:
    - Home
    - Product Detail
    - Cart
    - Checkout

❗ Vấn đề gặp phải

- Lạm dụng `push()`:
    - stack phình to
    - back button hành xử khó đoán
- Truyền dữ liệu sản phẩm:
    - constructor truyền quá nhiều thứ
- Khi mở từ link sản phẩm:
    - app không vào đúng màn hình

🛠 Cách áp dụng Slot 8

- **Named Routes** cho các màn hình chính
- Truyền arguments:
    - productId
- Hiểu **Navigator 2.0 concept**:
    - productId có → Product Detail
    - cart có item → Cart screen
- Chuẩn bị **deep link mở thẳng product**

PhucPT10@fpt.edu.vn

🎯 Mục tiêu đạt được

- User flow mạch lạc
- Có thể:
  - mở app từ link sản phẩm
  - quay lại đúng màn hình
- App sẵn sàng mở rộng:
  - wishlist
  - promotion

◈ TÌNH HUỐNG 3: APP NỘI BỘ – LOGIN → DASHBOARD

📍 Bối cảnh

- App cho nhân viên:
  - Login
  - Dashboard
  - Nhiều màn hình chức năng
- App có notification dẫn tới màn hình cụ thể

❗ Vấn đề gặp phải

- Sau login:
  - phải push nhiều lần
- Khi mở từ notification:
  - app luôn về Home
- Sinh viên không hiểu:
  - vì sao điều hướng sai

🛠 Cách áp dụng Slot 8

- Hiểu **Navigator 2.0 tư duy**:
  - loginState = false → Login
  - loginState = true → Dashboard
- Deep link:
  - nếu login rồi → mở thẳng màn hình feature
  - nếu chưa login → Login → redirect

🎯 Mục tiêu đạt được

- Luồng rõ ràng, đúng ngữ cảnh
- Không cần "vá" navigation
- Phù hợp app nội bộ, MVP startup

3 CASE STUDY – MVP STARTUP (MÔ PHỎNG)

📍 Bối cảnh

- Startup làm app chia sẻ nội dung
- Có:
  - Home
  - Detail
  - Profile
  - Link chia sẻ bài viết

❗ Vấn đề ban đầu

- Dùng push/pop tùy tiện
- Không mở được bài viết từ link
- Thêm màn hình là navigation rối

PhucPT10@fpt.edu.vn

🛠 Áp dụng Slot 8

- Tổ chức **Named Routes tập trung**
- Truyền dữ liệu qua arguments
- Hiểu Navigator 2.0 để:
  - quyết định màn hình dựa trên state
- Áp dụng Android Deep Linking:
  - link bài viết → mở đúng detail

✅ Kết quả

- Luồng rõ ràng
- MVP triển khai nhanh
- Sẵn sàng scale:
  - login
  - notification
  - marketing link

4 BÀI HỌC RÚT RA (LESSONS LEARNED)

🔑 Điều QUAN TRỌNG NHẤT

Navigation không phải là push/pop,
mà là thiết kế luồng người dùng.

⚠ Sai lầm sinh viên hay mắc

- Lạm dụng `Navigator.push()`
- Không hiểu bản chất route
- Nghĩ deep link là "phần phụ"
- Vá navigation khi app đã lớn

✅ Nguyên tắc cần duy trì lâu dài

- Luôn vẽ **user flow** trước khi code
- Tổ chức route rõ ràng từ sớm
- Tách navigation logic khỏi UI
- Nghĩ trước:
  - app mở từ đâu?
  - user vào màn hình nào đầu tiên?

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

PHẦN A. TƯ DUY NỀN TẢNG VỀ NAVIGATION TRONG FLUTTER

(Flutter Navigation Fundamentals)

I. TƯ DUY ĐIỀU HƯỚNG TRONG ỨNG DỤNG

◈ Navigation là gì trong app mobile?

Navigation là cách ứng dụng:

xác định **màn hình nào được hiển thị**
thứ tự các màn hình
và **cách quay lại / đi tiếp**

Navigation **không chỉ là "chuyển màn hình"**, mà là:

kiểm soát **luồng người dùng (user flow)**
phản ánh **trạng thái hiện tại của app**

◈ Vì sao Navigation ≠ chỉ là chuyển màn hình?

Chuyển màn hình chỉ là **hành động UI**
Navigation là:

quyết định **app đang ở trạng thái nào**
user **được phép đi đâu**
user **quay lại đâu**

👉 Ví dụ:

User chưa login → **không được vào Profile**
User mở app từ link → **không đi qua Home**

◈ Các khái niệm nền tảng

Screen / Page

Screen / Page: một màn hình UI mà user nhìn thấy
Trong Flutter:

thường là một `Widget`

❗ Sinh viên hay hiểu sai:

nghĩ screen = widget đơn lẻ
không hiểu screen nằm trong **luồng tổng thể**

Route

Route là:

đại diện logic cho **một màn hình**
có thể mang theo dữ liệu (arguments)

Route **không phải widget**
Widget chỉ là **nội dung hiển thị của route**

Navigation Stack

Là **ngăn xếp các route**
Route mới → push vào stack
Back → pop khỏi stack
Stack phản ánh **lịch sử điều hướng của user**

◈ So sánh app nhỏ vs app thật

App 1–2 màn hình:

push/pop là đủ

App nhiều màn hình:

cần **thiết kế luồng**
cần **quản lý stack**

PhucPT10@fpt.edu.vn

cần **kiểm soát entry point**

📌 **Slot 8 xuất hiện khi push/pop không còn đủ nữa**

II. MATERIALAPP & NAVIGATION ROOT

◈ Vai trò của MaterialApp

`MaterialApp` là **navigation root**

Là nơi Flutter:

- quản lý route
- quản lý navigator
- xử lý back button
- xử lý deep link

👉 Nếu navigation không rõ ràng ở đây → app sẽ rối về sau

◈ Các thuộc tính quan trọng

`home`

- màn hình mặc định
- chỉ phù hợp app rất nhỏ

`routes`

- map route name → widget
- đơn giản, dễ hiểu

`initialRoute`

- xác định **màn hình khởi đầu**
- quan trọng khi app có nhiều entry point

`onGenerateRoute`

- xử lý route động
- dùng khi cần truyền dữ liệu

◈ Lỗi tư duy phổ biến

Viết navigation rải rác trong UI

Không có "bản đồ màn hình"

Sửa 1 màn hình → vỡ luồng khác

📌 **Navigation phải được thiết kế từ root, không phải vá từ dưới lên**

III. NAMED ROUTES – ĐIỀU HƯỚNG CÓ CẤU TRÚC

◈ Named Route là gì?

Là cách **đặt tên cho màn hình**

Điều hướng bằng tên thay vì widget cụ thể

👉 Ví dụ tư duy:

"Đi tới `/profile`"

thay vì "mở widget ProfileScreen"

◈ Vì sao app lớn cần Named Routes?

App lớn cần:

- đọc code dễ
- biết màn hình nào tồn tại
- kiểm soát luồng từ nhiều nơi

Named Routes giúp:

giảm coupling

tăng khả năng maintain

PhucPT10@fpt.edu.vn

◈ So sánh

`Navigator.push()`:

- nhanh
- phù hợp demo

`Navigator.pushNamed()`:

- có cấu trúc
- phù hợp app thật

◈ Lỗi phổ biến

Sai route name → crash
Hard-code string → khó debug
Không quản lý tập trung

📌 Đây là **vấn đề kiến trúc**, không phải cú pháp

IV. PASSING ARGUMENTS GIỮA CÁC MÀN HÌNH

◈ Vì sao màn hình cần nhận dữ liệu?

Màn hình hiếm khi đứng một mình
Ví dụ:

- Detail cần id
- Profile cần user
- Checkout cần cart

◈ Các cách truyền arguments

Qua constructor

Rõ ràng
Phù hợp push trực tiếp
Dễ rối khi app lớn

Qua route settings

Phù hợp Named Routes
Tách UI khỏi logic điều hướng

◈ Tư duy đúng

Hỏi:

- "Màn hình này cần dữ liệu gì để tồn tại?"

Không truyền thừa
Không phụ thuộc màn hình trước đó

◈ Lỗi sinh viên hay gặp

`null` → quên truyền
Sai type → ép kiểu sai
UI crash → dữ liệu không đảm bảo

📌 Lỗi này là **lỗi tư duy dữ liệu**, không phải lỗi Flutter

V. NAVIGATOR 1.0 VS NAVIGATOR 2.0 (TƯ DUY KIẾN TRÚC)

◈ Hạn chế của Navigator 1.0

Điều hướng dựa trên **hành động**
Khó xử lý:

- login flow
- deep link
- refresh (web)

PhucPT10@fpt.edu.vn

◈ Navigator 2.0 giải quyết gì?

Điều hướng dựa trên **trạng thái**
Màn hình = kết quả của state

👉 Ví dụ:

`isLoggedIn = false` → Login
`isLoggedIn = true` → Home

◈ Các khái niệm cốt lõi

Declarative Navigation:
mô tả màn hình, không điều khiển từng bước
Router:
quyết định route từ state
Page stack as state:
stack là dữ liệu

◈ Lưu ý sư phạm

Không cần code sâu
Cần hiểu:
vì sao tồn tại
khi nào cần dùng

📌 Đây là **nền tảng cho GoRouter / AutoRoute**

VI. DEEP LINKING – APP & THẾ GIỚI BÊN NGOÀI

◈ Deep Linking là gì?

Mở app từ **link bên ngoài**
Không đi theo luồng thông thường

◈ Khác navigation nội bộ ở đâu?

Navigation nội bộ:
user đang trong app
Deep link:
app có thể:
đang tắt
đang nền
đang mở

◈ Tình huống thực tế

Click link sản phẩm
Notification
Chia sẻ nội dung

◈ Vai trò trong app thương mại

Marketing
Retention
User experience

📌 App thật **bắt buộc phải nghĩ tới deep link**

VII. LỖI TƯ DUY NAVIGATION PHỔ BIẾN

✕ Push / pop không kiểm soát

Nguyên nhân: không có sơ đồ luồng
Hậu quả: stack rối, back sai

PhucPT10@fpt.edu.vn

❌ Navigation gắn chặt vào UI
Nguyên nhân: code nhanh
Hậu quả: khó refactor

❌ Không thiết kế luồng từ đầu
Nguyên nhân: làm app như bài tập
Hậu quả: vá navigation khi app lớn

VIII. ĐỊNH HƯỚNG KIẾN TRÚC NAVIGATION CHO APP LỚN

◈ Navigation liên quan trực tiếp tới:
State management
Authentication
Role / permission

◈ Chuẩn bị cho:
Declarative UI
Router
App quy mô công nghiệp
📌 Slot 8 là **bước đặt móng**, không phải đích đến

IX. TỔNG KẾT PHẦN A

Navigation = xương sống của app
Code chạy ≠ kiến trúc đúng
Slot 8 = chuyển từ **học Flutter → làm app thật**

PHẦN B. LIÊN HỆ VỚI DART & KIẾN TRÚC ỨNG DỤNG

I. DART & NAVIGATION LOGIC

◈ Vai trò của Dart
`class`: mô tả dữ liệu truyền giữa màn hình
`enum`: mô tả trạng thái điều hướng
object arguments: giúp code rõ ràng, an toàn
👉 Navigation tốt cần **Dart typed rõ ràng**

II. STATE & NAVIGATION (NHẬN THỨC BAN ĐẦU)

Navigation chính là **một dạng state**
UI hiển thị = state hiện tại
👉 Đây là nền tảng để học:
Provider
Bloc
Riverpod
Declarative routing

III. TỔNG KẾT SLOT 8

🔍 Nếu chỉ giữ lại 3 ý:
Navigation là thiết kế luồng, không phải push/pop
Named Routes giúp app có cấu trúc & dễ mở rộng
Navigator 2.0 & Deep Link là bắt buộc cho app thật

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. Kỹ thuật: Named Routes & Route Arguments

Kinh nghiệm thực tế khi sử dụng trong dự án Flutter

Dùng ở đâu trong app thật?

App nhỏ → trung bình (10–20 màn hình)
App CRUD, admin panel, app sinh viên
Các màn hình có thể truy cập từ nhiều nơi (Home → Detail, Search → Detail)

Ngoài lý thuyết phát sinh gì?

Số lượng route tăng nhanh → khó nhớ tên
Arguments truyền sai kiểu → crash runtime
Không biết route này được gọi từ đâu

Bối cảnh dự án

App quản lý sinh viên: Home → StudentDetail → EditStudent

Vấn đề navigation

Truyền object Student qua Named Route
Một số nơi chỉ truyền id, nơi khác truyền full object

Vì sao chọn Named Routes?

Cần route name rõ ràng
Dễ cấu hình ban đầu trong `MaterialApp`

Lỗi thường gặp

Lỗi 1: Crash khi nhận arguments

Mô tả: App chạy bình thường, nhưng crash khi mở màn hình
Nguyên nhân gốc:
Ép kiểu trực tiếp `as Student`
Route được gọi từ nhiều nơi với arguments khác nhau

Ví dụ:

```
final student = ModalRoute.of(context)!.settings.arguments as Student;
```

Khắc phục:
Dùng class wrapper cho arguments
Check null & type

Phòng tránh:
Quy ước 1 route = 1 loại arguments
Không dùng `dynamic`

Lỗi 2: Route name trùng / khó đọc

Nguyên nhân: đặt tên tuỳ hứng: `/detail`, `/detail2`

Khắc phục & phòng tránh:
Tạo file `app_routes.dart`
Dùng hằng số:

```
static const studentDetail = '/student/detail';
```

Tips & Tricks

Prefix route theo feature: `/student/detail`
Log route khi push (debug navigation)
Viết test: `pumpWidget` + `tester.tap`

Best Practices

Nên dùng Named Routes khi:
App nhỏ – trung bình

PhucPT10@fpt.edu.vn

Luồng đơn giản

❌ **Không nên dùng khi**:

Luồng phụ thuộc state (login/logout)

Deep link phức tạp

🔄 Liên hệ thực tế

Trước: route rải rác, truyền arguments lung tung

Sau: central route config → dễ maintain

🎓 **Bài học rút ra**

Named Routes **không sai**, nhưng **không đủ** cho app lớn.

⚠️ **Cảnh báo tư duy sai**

"Truyền được object là xong"

🔷 2. Kỹ thuật: Navigator 1.0 – Push / Pop / Stack

✅ Kinh nghiệm thực tế

Dùng ở đâu?

App MVP

Luồng tuyến tính: List → Detail → Edit

📍 **Bối cảnh dự án**

App bán hàng mini: Home → Product → Checkout

❗ **Vấn đề**

Back stack sai sau login

Nhấn back quay lại màn hình không mong muốn

🎯 **Lý do chọn Navigator 1.0**

Nhanh, dễ code, quen thuộc

⚠️ Lỗi thường gặp

❌ *Lỗi 1: Back stack sai sau login*

🧪 *Tình huống*:

Navigator.push(context, LoginPage());

→ Login xong back về Login

🔍 **Nguyên nhân**:

Không hiểu stack

🛠 **Khắc phục**:

Navigator.pushReplacement(...);

🚫 **Phòng tránh**:

Vẽ stack trước khi code

💡 Tips & Tricks

Dùng `pushReplacement`, `pushAndRemoveUntil` đúng ngữ cảnh

Debug stack bằng logging

🛡️ Best Practices

Navigator 1.0 **đủ dùng khi**:

Luồng đơn giản

Không deep link

Bắt đầu không đủ khi:

Có auth flow

Restore state

🎓 **Bài học**

PhucPT10@fpt.edu.vn

Hiểu stack quan trọng hơn nhớ hàm.

⚠️ **Cảnh báo**

"Cứ push là được"

🔷 3. Nhận thức kiến trúc: Navigator 2.0 (State-based Navigation)

✅ Kinh nghiệm thực tế

Dùng khi nào?

App lớn
Login / Logout
Deep link

📍 **Bối cảnh**

App có Auth + Profile + Admin

❗ **Vấn đề**

Push/pop không kiểm soát được khi state đổi

🎯 **Vì sao cần Navigator 2.0?**

UI phản ánh state
Không phụ thuộc thao tác tay

⚠️ Lỗi nhận thức phổ biến

❌ *Lỗi: Nghĩ Navigator 2.0 chỉ là API mới*

🔍 **Nguyên nhân**:

Không hiểu bản chất *state-driven*

🛠 **Khắc phục**:

Nghĩ: *State → Navigation*

💡 Tips

Vẽ sơ đồ state trước
Tách Router khỏi UI

🛡 Best Practices

Navigation không nằm trong button
Auth quyết định màn hình

🎓 **Bài học**

Navigator 2.0 là **tư duy**, không phải API.

🔷 4. Kỹ thuật: Điều hướng theo User Flow

✅ Kinh nghiệm thực tế

📍 **Bối cảnh**

Onboarding → Login → Home → Feature

❗ **Vấn đề**

Code theo màn hình, không theo flow

🎯 **Giải pháp**

Xác định flow trước
Mỗi flow = 1 nhánh navigation

⚠️ Lỗi phổ biến

❌ Nhảy màn hình tuỳ tiện

🔍 **Nguyên nhân**:

Không vẽ flow

🛠 **Khắc phục**:

PhucPT10@fpt.edu.vn

Diagram user flow

🛡️ Best Practices

Flow > Screen
Navigation phục vụ trải nghiệm

🎓 **Bài học**

App đúng flow = app dễ maintain

🔷 5. Kỹ thuật: Android Deep Linking (Thực tế)

✅ Kinh nghiệm thực tế

📍 **Bối cảnh**

Mở link `/product/123` từ email

❗ **Vấn đề**

App mở sai màn hình
Chưa login

🎯 **Giải pháp**

Kết hợp deep link + auth state

⚠️ Lỗi phổ biến

❌ Deep link mở thẳng màn hình nhạy cảm

🔍 **Nguyên nhân**:

Không check state

🛠️ **Khắc phục**:

Redirect về login → resume link

🛡️ Best Practices

Deep link = input bên ngoài
Luôn validate

🎓 **Bài học**

Deep link không phải shortcut, mà là entry point.

🎯 KẾT LUẬN CHO SINH VIÊN

Navigation sách ≠ navigation dự án thật
Thiết kế flow **trước khi code**
Hiểu *vì sao* dùng kỹ thuật, không chỉ *dùng như thế nào*

Nếu chỉ nhớ 2 ý: 1️⃣ Navigation là kiến trúc, không phải thao tác 2️⃣ Sai navigation = app chạy nhưng sai UX

PhucPT10@fpt.edu.vn

EXERCISE

BÀI TẬP TỔNG HỢP 1

(Navigation – Named Routes – Navigator 1.0/2.0 – Deep Linking)

1 TỔNG QUAN DỰ ÁN

Tên dự án

Student Manager App

Mô tả ngắn gọn

Ứng dụng Flutter quản lý danh sách sinh viên đơn giản, gồm nhiều màn hình và luồng điều hướng rõ ràng:

Xem danh sách sinh viên
Xem chi tiết từng sinh viên
Thêm / chỉnh sửa thông tin sinh viên
Mở trực tiếp màn hình chi tiết thông qua deep link

Ứng dụng **không tập trung UI**, mà tập trung vào **thiết kế và triển khai navigation đúng tư duy app thật**.

Mục tiêu học tập

Sau dự án này, sinh viên có thể:

Hiểu bản chất **route – navigation stack**
Sử dụng **Named Routes & Arguments** đúng mục đích
Làm chủ **Navigator 1.0 (push / pop / stack)**
Có **nhận thức nền tảng về Navigator 2.0 (routing theo trạng thái)**
Hiểu deep link hoạt động như thế nào trong app Android thật

Đối tượng phù hợp

Sinh viên đã học xong Flutter Fundamentals
Trình độ trung bình – yếu
Đã biết StatelessWidget / StatefulWidget / setState

2 CHỨC NĂNG CHÍNH CỦA ỨNG DỤNG

Danh sách màn hình (Screens)

Screen	Chức năng	Navigation sử dụng
HomeScreen	Trang vào app	Named Route
StudentListScreen	Hiển thị danh sách sinh viên	pushNamed
StudentDetailScreen	Xem chi tiết sinh viên	pushNamed + arguments
StudentFormScreen	Thêm / sửa sinh viên	push + pop

👉 **Mỗi màn hình gắn với một vai trò rõ ràng trong user flow**

User Flow chính

Home → Danh sách sinh viên
Chọn sinh viên → Xem chi tiết
Từ chi tiết → Quay lại danh sách
Từ danh sách → Thêm sinh viên → Quay lại danh sách
Mở app bằng deep link → vào thẳng màn hình chi tiết

3 CÔNG NGHỆ & CÔNG CỤ SỬ DỤNG

Flutter SDK (stable)
Dart (null safety)
Android Studio
Android Emulator hoặc thiết bị thật
Material Widgets
Android Deep Link (AndroidManifest.xml)

PhucPT10@fpt.edu.vn

4 CẤU TRÚC THƯ MỤC DỰ ÁN

```
lib/
 ├─ main.dart
 ├─ routes/
 │   └─ app_routes.dart
 ├─ screens/
 │   ├─ home_screen.dart
 │   ├─ student_list_screen.dart
 │   ├─ student_detail_screen.dart
 │   └─ student_form_screen.dart
 ├─ models/
 │   └─ student.dart
 └─ widgets/
     └─ student_tile.dart
```

5 DANH SÁCH CÁC BƯỚC THỰC HIỆN (TÓM TẮT)

Tạo project Flutter
Thiết lập cấu trúc thư mục
Tạo model `Student`
Khai báo Named Routes trong `MaterialApp`
Xây dựng HomeScreen
Xây dựng StudentListScreen
Điều hướng sang DetailScreen bằng arguments
Xử lý pop & back stack
Thêm màn hình Form (Create/Edit)
Nhận thức Navigator 2.0 (phân tích luồng)
Cấu hình Android Deep Link
Test toàn bộ luồng navigation

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

0. Chuẩn bị

0.1. Tạo project Flutter mới

Mở **Android Studio** (hoặc VS Code).
Chọn **New Flutter Project**.
Đặt tên: `student_manager_app`.
Tạo xong, đợi Flutter tải dependencies.

Sau khi tạo xong, chạy thử:

```
flutter run
```

Nếu app mặc định "Counter" chạy được là môi trường OK.

1. Thiết lập cấu trúc thư mục

1.1. Tạo các thư mục trong `lib/`

Trong thư mục `lib`, tạo các thư mục con như sau:

Text

```
lib/
 ├─ main.dart
 ├─ routes/
 │   └─ app_routes.dart
 ├─ screens/
 │   ├─ home_screen.dart
```


PhucPT10@fpt.edu.vn

```
│   ├─ student_list_screen.dart
│   ├─ student_detail_screen.dart
│   └─ student_form_screen.dart
├─ models/
│   └─ student.dart
└─ widgets/
    └─ student_tile.dart
```

Giải thích tư duy tách thư mục

- `screens/`: Mỗi file = 1 màn hình. Navigation sẽ đi giữa các màn hình này.
- `routes/`: Nơi định nghĩa danh sách routes, tránh viết route string rải rác.
- `models/`: Chứa các class dữ liệu (ở đây là `Student`), được truyền qua lại giữa các màn hình.
- `widgets/`: Chứa các widget con, **không nên** chứa logic điều hướng.

---

2. Code từng file (theo thứ tự dễ → khó)

2.1. File `models/student.dart`

Mục tiêu: có model `Student` để truyền làm arguments giữa các màn hình.

Tạo file: `lib/models/student.dart`

lib/models/student.dart

```dart
class Student {
  final String id;
  final String name;
  final int age;

  Student({
    required this.id,
    required this.name,
    required this.age,
  });

  // Optional: tiện cho debug / log
  @override
  String toString() {
    return 'Student(id: $id, name: $name, age: $age)';
  }

  // Copy with để dễ chỉnh sửa (edit student)
  Student copyWith({
    String? id,
    String? name,
    int? age,
  }) {
    return Student(
      id: id ?? this.id,
      name: name ?? this.name,
      age: age ?? this.age,
    );
  }
}
```

Giải thích:

- `id`: string (giả sử là mã sinh viên).

PhucPT10@fpt.edu.vn

name: tên sinh viên.
age: tuổi, kiểu `int`.
`copyWith`: sau này dùng khi sửa thông tin sinh viên (edit).

2.2. File `routes/app_routes.dart`
Mục tiêu: định nghĩa **hằng số route** để tránh sai chính tả.
Tạo file: `lib/routes/app_routes.dart`
lib/routes/app_routes.dart

```
class AppRoutes {
  static const String home = '/';
  static const String studentList = '/students';
  static const String studentDetail = '/students/detail';
  static const String studentForm = '/students/form';
}
```

Giải thích:
Dùng `static const` để:
Có gợi ý autocomplete.
Tránh gõ nhầm string.
Dễ refactor khi đổi tên route.

2.3. File `widgets/student_tile.dart`
Mục tiêu: tạo widget hiển thị mỗi sinh viên trong list.
Tạo file: `lib/widgets/student_tile.dart`
lib/widgets/student_tile.dart

```
import 'package:flutter/material.dart';
import '../models/student.dart';

class StudentTile extends StatelessWidget {
  final Student student;
  final VoidCallback? onTap;

  const StudentTile({
    Key? key,
    required this.student,
    this.onTap,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return ListTile(
      title: Text(student.name),
      subtitle: Text('ID: ${student.id} | Tuổi: ${student.age}'),
      trailing: const Icon(Icons.chevron_right),
      onTap: onTap,
    );
  }
}
```

Giải thích:
`StudentTile` chỉ có nhiệm vụ **render UI** 1 dòng sinh viên.
Không xử lý navigation bên trong, mà nhận `onTap` từ màn hình cha.

PhucPT10@fpt.edu.vn

2.4. File `screens/home_screen.dart`

Mục tiêu: màn hình vào app, có nút đi đến danh sách sinh viên.

Tạo file: `lib/screens/home_screen.dart`

lib/screens/home_screen.dart

```
import 'package:flutter/material.dart';
import '../routes/app_routes.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({Key? key}) : super(key: key);

  void _goToStudentList(BuildContext context) {
    // Sử dụng Named Route
    Navigator.pushNamed(context, AppRoutes.studentList);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Student Manager - Home'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () => _goToStudentList(context),
          child: const Text('Xem danh sách sinh viên'),
        ),
      ),
    );
  }
}
```

Giải thích:

`Navigator.pushNamed(context, AppRoutes.studentList);`

Dùng `pushNamed` để điều hướng bằng **tên route**.

Tên route lấy từ `AppRoutes`.

2.5. File `screens/student_list_screen.dart`

Mục tiêu:

Hiển thị danh sách sinh viên giả lập (in-memory).

Khi bấm vào 1 sinh viên → đi đến `StudentDetailScreen` với **arguments** là `Student`.

Có nút FAB để mở form thêm mới.

Tạo file: `lib/screens/student_list_screen.dart`

lib/screens/student_list_screen.dart

```
import 'package:flutter/material.dart';
import '../models/student.dart';
import '../widgets/student_tile.dart';
import '../routes/app_routes.dart';
import 'student_form_screen.dart';

class StudentListScreen extends StatefulWidget {
```


PhucPT10@fpt.edu.vn

```
  const StudentListScreen({Key? key}) : super(key: key);

  @override
  State<StudentListScreen> createState() => _StudentListScreenState();
}

class _StudentListScreenState extends State<StudentListScreen> {
  // Danh sách sinh viên giả lập (trong bộ nhớ)
  final List<Student> _students = [
    Student(id: 'S001', name: 'Nguyễn Văn A', age: 20),
    Student(id: 'S002', name: 'Trần Thị B', age: 21),
    Student(id: 'S003', name: 'Lê Văn C', age: 19),
  ];

  void _goToDetail(Student student) {
    // Navigator 1.0 + Named Route + arguments
    Navigator.pushNamed(
      context,
      AppRoutes.studentDetail,
      arguments: student,
    );
  }

  Future<void> _goToForm() async {
    // Mở form để tạo mới
    // Ở đây dùng push (không named) để sinh viên thấy sự khác nhau
    final result = await Navigator.push<Student?>(
      context,
      MaterialPageRoute(
        builder: (ctx) => const StudentFormScreen(),
      ),
    );

    // Khi form pop về, nếu có student mới thì thêm vào list
    if (result != null) {
      setState(() {
        _students.add(result);
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Danh sách sinh viên'),
      ),
      body: ListView.builder(
        itemCount: _students.length,
        itemBuilder: (context, index) {
          final student = _students[index];
          return StudentTile(
            student: student,
```


PhucPT10@fpt.edu.vn

```
            onTap: () => _goToDetail(student),
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _goToForm,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Giải thích từng phần:

`_students`: list giả để demo. Không dùng database để sinh viên tập trung vào navigation.

`_goToDetail(student)`:

`Navigator.pushNamed(context, AppRoutes.studentDetail, arguments: student);`

Đây là **Navigator 1.0** với **Named Route** và **arguments** kiểu `Student`.

`_goToForm()`:

Dùng `Navigator.push` với `MaterialPageRoute` (không named).

`await` để nhận **kết quả trả về** từ form (ở đây là `Student?`).

Nếu kết quả không null → `setState` để thêm vào danh sách.

2.6. File `screens/student_detail_screen.dart`

Mục tiêu:

Nhận `Student` từ `arguments`.

Hiển thị thông tin chi tiết.

Có nút Back → `Navigator.pop(context)`.

Tạo file: `lib/screens/student_detail_screen.dart`

lib/screens/student_detail_screen.dart

```
import 'package:flutter/material.dart';
import '../models/student.dart';

class StudentDetailScreen extends StatelessWidget {
  const StudentDetailScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    // Lấy arguments từ ModalRoute
    final args = ModalRoute.of(context)?.settings.arguments;

    // Kiểm tra kiểu dữ liệu để tránh lỗi cast
    if (args == null || args is! Student) {
      // Nếu vào đây do deep link hoặc lỗi, ta hiển thị thông báo
      return Scaffold(
        appBar: AppBar(
          title: const Text('Chi tiết sinh viên'),
        ),
        body: const Center(
          child: Text('Không có dữ liệu sinh viên (arguments null hoặc sai
kiểu).'),
```


PhucPT10@fpt.edu.vn

```
        ),
      );
    }

    final student = args;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Chi tiết sinh viên'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text('ID: ${student.id}', style:
Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 8),
            Text('Tên: ${student.name}', style:
Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 8),
            Text('Tuổi: ${student.age}', style:
Theme.of(context).textTheme.titleMedium),
            const Spacer(),
            Center(
              child: ElevatedButton(
                onPressed: () {
                  // Pop để quay wieder lại list
                  Navigator.pop(context);
                },
                child: const Text('Quay lại danh sách'),
              ),
            )
          ],
        ),
      ),
    );
  }
}
```

Giải thích quan trọng:

`ModalRoute.of(context)?.settings.arguments;`

Đây là cách **chuẩn** để lấy arguments.

Kiểm tra:

`args == null` → tránh lỗi null.

`args is! Student` → tránh `TypeError` khi cast sai.

2.7. File `screens/student_form_screen.dart`

Mục tiêu:

Màn hình thêm mới (create) sinh viên.

Dùng `TextEditingController` và `Form`.

Khi bấm Save:

Validate data.

Tạo object Student.

`Navigator.pop(context, student);` trả kết quả về cho màn hình list.

Tạo file: `lib/screens/student_form_screen.dart`

lib/screens/student_form_screen.dart

```
import 'package:flutter/material.dart';
import '../models/student.dart';

class StudentFormScreen extends StatefulWidget {
  const StudentFormScreen({Key? key}) : super(key: key);

  @override
  State<StudentFormScreen> createState() => _StudentFormScreenState();
}

class _StudentFormScreenState extends State<StudentFormScreen> {
  final _formKey = GlobalKey<FormState>();
  final _idController = TextEditingController();
  final _nameController = TextEditingController();
  final _ageController = TextEditingController();

  @override
  void dispose() {
    // Giải phóng controller để tránh leak memory
    _idController.dispose();
    _nameController.dispose();
    _ageController.dispose();
    super.dispose();
  }

  void _saveStudent() {
    // Kiểm tra hợp lệ form
    if (_formKey.currentState?.validate() != true) {
      return;
    }

    final id = _idController.text.trim();
    final name = _nameController.text.trim();
    final age = int.tryParse(_ageController.text.trim()) ?? 0;

    final newStudent = Student(id: id, name: name, age: age);

    // Pop và trả newStudent về cho màn hình trước
    Navigator.pop(context, newStudent);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Thêm sinh viên'),
```

```
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Form(
        key: _formKey,
        child: Column(
          children: [
            TextFormField(
              controller: _idController,
              decoration: const InputDecoration(
                labelText: 'Mã sinh viên',
              ),
              validator: (value) {
                if (value == null || value.trim().isEmpty) {
                  return 'Vui lòng nhập mã sinh viên';
                }
                return null;
              },
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _nameController,
              decoration: const InputDecoration(
                labelText: 'Họ tên',
              ),
              validator: (value) {
                if (value == null || value.trim().isEmpty) {
                  return 'Vui lòng nhập họ tên';
                }
                return null;
              },
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _ageController,
              decoration: const InputDecoration(
                labelText: 'Tuổi',
              ),
              keyboardType: TextInputType.number,
              validator: (value) {
                final text = value?.trim() ?? '';
                final age = int.tryParse(text);
                if (text.isEmpty) {
                  return 'Vui lòng nhập tuổi';
                }
                if (age == null || age <= 0) {
                  return 'Tuổi không hợp lệ';
                }
                return null;
              },
            ),
            const SizedBox(height: 24),
            Row(
```


PhucPT10@fpt.edu.vn

```
                mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                children: [
                  ElevatedButton(
                    onPressed: _saveStudent,
                    child: const Text('Lưu'),
                  ),
                  OutlinedButton(
                    onPressed: () {
                      // Không lưu, chỉ pop về (trả null)
                      Navigator.pop(context);
                    },
                    child: const Text('Hủy'),
                  ),
                ],
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Giải thích:

`Form + GlobalKey<FormState>` → dùng để validate input.

`Navigator.pop(context, newStudent);`

Trả lại `Student` cho màn hình trước (`StudentListScreen`).

Nếu bấm Hủy → `Navigator.pop(context);` không truyền gì (null).

2.8. File `main.dart` – root navigation

Mục tiêu: tạo `MaterialApp`, khai báo `routes`, `initialRoute`, và `onGenerateRoute` để chuẩn bị cho deep link / Navigator 2.0 sau này.

Mở và sửa `lib/main.dart`:

lib/main.dart

```
import 'package:flutter/material.dart';
import 'routes/app_routes.dart';
import 'screens/home_screen.dart';
import 'screens/student_list_screen.dart';
import 'screens/student_detail_screen.dart';

void main() {
  runApp(const StudentManagerApp());
}

class StudentManagerApp extends StatelessWidget {
  const StudentManagerApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Student Manager',
```

```
      debugShowCheckedModeBanner: false,
      initialRoute: AppRoutes.home,
      // Định nghĩa routes tĩnh
      routes: {
        AppRoutes.home: (context) => const HomeScreen(),
        AppRoutes.studentList: (context) => const StudentListScreen(),
        AppRoutes.studentDetail: (context) => const StudentDetailScreen(),
        // Form screen đang dùng push(MaterialPageRoute) nên không cần khai
báo ở đây
      },

      // onGenerateRoute: chuẩn bị cho các trường hợp route không có trong
routes map
      onGenerateRoute: (settings) {
        // Ví dụ: sau này xử lý deep link /students/detail?id=S001 ở đây
        // Ở bài này ta chỉ log cho sinh viên thấy flow
        debugPrint('onGenerateRoute được gọi với: ${settings.name}');
        return null; // Trả null để MaterialApp dùng tiếp onUnknownRoute (nếu
có)
      },

      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
    );
  }
}
```

Giải thích:

- `initialRoute: AppRoutes.home` → app mở vào `HomeScreen`.
- `routes: {...}`:
  - Map giữa `String routeName` và `WidgetBuilder`.
- `onGenerateRoute`:
  - Tạm thời chưa xử lý gì, nhưng cho sinh viên thấy đây là nơi sẽ xử lý Navigator 2.0 / deep link custom sau này.

3. Chạy thử Navigator 1.0 & Named Routes

3.1. Các bước test luồng cơ bản

- `flutter run`.
- Màn hình `HomeScreen` → bấm "Xem danh sách sinh viên".
- Ở `StudentListScreen`:
  - Tap 1 sinh viên → vào `StudentDetailScreen`.
  - Bấm "Quay lại danh sách" → về `StudentListScreen`.
- Ở `StudentListScreen`:
  - Bấm nút + → mở `StudentFormScreen`.
  - Nhập thông tin hợp lệ → Lưu.
  - Về lại `StudentListScreen` → thấy sinh viên mới thêm.

3.2. Lỗi thường gặp và cách xử lý

Sai route name

PhucPT10@fpt.edu.vn

Ví dụ: `Navigator.pushNamed(context, '/student');` thay vì `AppRoutes.studentList`.

Cách tránh:

Luôn dùng `AppRoutes.xxx`.

arguments null hoặc sai kiểu

Quên truyền `arguments` khi `pushNamed`.

Giải pháp:

Kiểm tra `args == null || args is! Student`.

Context sai

Gọi `Navigator.pushNamed` trong hàm `initState` dùng `context` của `State` có thể gây warn.

Khuyến nghị: chỉ navigate sau khi build xong, hoặc trong sự kiện user (onPressed, onTap).

4. Nhận thức nền tảng về Navigator 2.0 (không code nhiều)

Mức này chỉ **giải thích tư duy**, không bắt sinh viên code phức tạp.

4.1. Navigator 1.0 – điều hướng bằng hành động

Em tưởng tượng một chồng giấy (stack):

`push` = đặt thêm 1 tờ giấy lên trên.

`pop` = lấy tờ giấy trên cùng ra.

Khi em bấm:

`Navigator.pushNamed(context, AppRoutes.studentDetail, arguments: student);`

App **thêm** màn hình `Detail` lên trên.

Khi em bấm `Navigator.pop(context);`

App **lấy** màn hình `Detail` ra, quay lại `List`.

4.2. Navigator 2.0 – điều hướng bằng trạng thái (state)

Thay vì "bảo Navigator: push / pop", em mô tả **trạng thái**:

"App đang ở màn hình list".

"App đang ở màn hình detail của student S001".

Framework sẽ nhìn vào state này và tự dựng **danh sách pages** tương ứng.

Deep link:

URL: `myapp://students/detail/S001`.

Navigator 2.0: khi nhận link, nó tạo state tương ứng (đang xem chi tiết `S001`), sau đó UI build theo state.

Ở bài này ta chưa code Navigator 2.0 đầy đủ, chỉ giúp sinh viên **đặt câu hỏi** và hình dung.

5. Deep Linking Android (cơ bản)

Mục tiêu: sinh viên **hiểu cơ chế**, có một cấu hình mẫu để thử.

5.1. Mở file AndroidManifest

Đường dẫn:

```Text
android/app/src/main/AndroidManifest.xml
```

Trong thẻ `<activity ...>` của MainActivity (chứa `android:name=".MainActivity"`), thêm `intent-filter` sau (bên trong `<activity>`):

```XML
<intent-filter>
```


PhucPT10@fpt.edu.vn

```
    <action android:name="android.intent.action.VIEW" />

    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />

    <!-- Scheme và host là tuỳ chọn thiết kế -->
    <data
        android:scheme="studentapp"
        android:host="student"
        android:pathPrefix="/detail" />
</intent-filter>
```

Giải thích:

- `android:scheme="studentapp"`: giống như `https`, `mailto`,... nhưng do ta tự đặt.
- `android:host="student"`: giống tên miền (vd: `google.com`).
- `android:pathPrefix="/detail"`: đường dẫn phía sau.

Ví dụ deep link:
`studentapp://student/detail?id=S001`

5.2. Test deep link với ADB (emulator)

Mở terminal:

```bash
adb shell am start \
  -a android.intent.action.VIEW \
  -d "studentapp://student/detail?id=S001"
```

Hiện tại app của ta **chưa parse query** `id=S001`, nhưng:

- Android sẽ mở app `Student Manager`.
- App sẽ được gọi theo `onGenerateRoute` (nếu cấu hình thêm).
- Sinh viên hiểu: deep link là "cửa" để mở app vào 1 context nào đó.

Bài tập nâng cao:
Dùng `onGenerateRoute` để đọc `settings.name`, split param,
tạo `StudentDetailScreen` tương ứng.

6. Ôn lại luồng User Flow

- Mở app → `HomeScreen`.
- Bấm nút → `StudentListScreen` (pushNamed).
- Chọn 1 dòng sinh viên → `StudentDetailScreen` (pushNamed + arguments).
- Bấm "Quay lại" → `Navigator.pop` → về List.
- Từ List:
  - Bấm + → `StudentFormScreen` (push).
  - Nhập, Lưu → `Navigator.pop(context, newStudent)` → List nhận student mới, `setState` hiển thị.

7 KIỂM TRA & CHẠY ỨNG DỤNG

- `flutter run`
- Test:
  - push / pop
  - back button
  - deep link

PhucPT10@fpt.edu.vn

8 CÁC LỖI PHỔ BIẾN & CÁCH XỬ LÝ

Sai route name → dùng hằng số
Stack phình to → kiểm soát push
Mất arguments → kiểm tra lifecycle
Context sai → không navigate trong build

9 KẾT QUẢ ĐẠT ĐƯỢC SAU DỰ ÁN

Sinh viên:

Thiết kế được navigation flow
Hiểu rõ stack
Sẵn sàng học routing nâng cao

10 BÀI TẬP MỞ RỘNG (KHUYẾN NGHỊ)

Thêm màn hình Edit Student
Deep link mở form chỉnh sửa
Thêm confirm dialog trước khi pop
Vẽ sơ đồ navigation flow
Phân tích nếu dùng Navigator 2.0
Thiết kế được navigation flow
Hiểu rõ stack
Sẵn sàng học routing nâng cao

---

10 BÀI TẬP MỞ RỘNG (KHUYẾN NGHỊ)

Thêm màn hình Edit Student
Deep link mở form chỉnh sửa
Thêm confirm dialog trước khi pop
Vẽ sơ đồ navigation flow
Phân tích nếu dùng Navigator 2.0

🎓 THÔNG ĐIỆP CUỐI

App chạy được ≠ App thiết kế đúng
Navigation là nền móng cho app lớn.

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

NHÓM 1 – THUẬT NGỮ CỐT LÕI VỀ NAVIGATION

Thuật ngữ	Giải thích dễ hiểu	Sinh viên hay hiểu sai	Liên hệ app thật
Navigation	Cách người dùng **di chuyển giữa các màn hình** trong app	Nghĩ navigation chỉ là `push()` sang màn hình khác	Luồng: Home → List → Detail → Back
Screen	Một màn hình UI hoàn chỉnh (thường là 1 Widget lớn)	Nhầm Screen với Widget nhỏ	HomeScreen, DetailScreen
Page	Khái niệm logic đại diện cho 1 màn hình trong navigation	Nghĩ Page = Screen 100%	Navigator 2.0 dùng Page
Route	Con đường dẫn đến một màn hình	Nghĩ Route chỉ là String	Route chứa Screen + dữ liệu
Navigation Stack	Danh sách các màn hình đang mở theo thứ tự	Không hình dung được stack	Back = pop màn hình trên cùng
Push	Đưa màn hình mới lên stack	Push xong quên pop	Mở Detail từ List
Pop	Quay lại màn hình trước	Pop nhầm context	Nhấn nút Back

NHÓM 2 – NAMED ROUTES & ROUTE ARGUMENTS

Thuật ngữ	Giải thích dễ hiểu	Sinh viên hay hiểu sai	Liên hệ app thật
Named Route	Route được đặt tên để quản lý tập trung	Dùng cho app nhỏ cũng bắt buộc	App nhiều màn hình
Route Name	Chuỗi định danh route (vd: `/detail`)	Viết sai nhưng không biết	Debug khó
routes (MaterialApp)	Map khai báo route tĩnh	Viết rải rác nhiều nơi	Quản lý navigation
initialRoute	Màn hình mở đầu của app	Nhầm với `home`	Login / Splash
onGenerateRoute	Hàm tạo route động	Nghĩ là nâng cao không cần	Truyền arguments
Route Arguments	Dữ liệu gửi sang màn hình khác	Truyền Map tùy ý	Gửi Student sang Detail
RouteSettings	Nơi chứa name & arguments	Không biết tồn tại	Debug route

NHÓM 3 – NAVIGATOR 1.0

Thuật ngữ	Giải thích dễ hiểu	Sinh viên hay hiểu sai	Liên hệ app thật
Navigator	Bộ điều khiển navigation	Nghĩ là Widget UI	Quản lý stack
Navigator 1.0	Điều hướng kiểu imperative (ra lệnh)	Không biết có version 2.0	App vừa – nhỏ
Imperative Navigation	Ra lệnh: push / pop	Dùng cho mọi trường hợp	Demo nhỏ
BuildContext	Ngữ cảnh của widget	Dùng sai context	Crash navigation
Back Stack	Lịch sử màn hình	Không kiểm soát	Back sai màn

NHÓM 4 – NAVIGATOR 2.0 (TƯ DUY)

Thuật ngữ	Giải thích dễ hiểu	Sinh viên hay hiểu sai	Liên hệ app thật
Navigator 2.0	Điều hướng theo trạng thái	Nghĩ phải code phức tạp	App lớn
Declarative Navigation	UI phụ thuộc state	Nhầm với setState	Auth flow
Page Stack as State	Stack là kết quả của state	Khó hình dung	Login / Logout

PhucPT10@fpt.edu.vn

Router	Bộ điều phối route	Nghĩ chỉ web mới có	Deep link
State-driven Navigation	Điều hướng theo state	Push thủ công	App ổn định

NHÓM 5 – DEEP LINKING

Thuật ngữ	Giải thích dễ hiểu	Sinh viên hay hiểu sai	Liên hệ app thật
Deep Link	Mở app vào màn hình cụ thể	Chỉ là link web	Mở product detail
External Navigation	Điều hướng từ bên ngoài app	Nhầm với push nội bộ	Notification
Android Intent	Cơ chế Android mở app	Không cần biết	Deep link hoạt động
App Link	Deep link chuẩn Android	Bỏ qua khi làm MVP	Chia sẻ link
Initial Route Handling	Xử lý màn hình khi app mở	Không thiết kế	App mở sai màn

NHÓM 6 – DART & CẤU TRÚC HỖ TRỢ NAVIGATION

Thuật ngữ	Giải thích dễ hiểu	Sinh viên hay hiểu sai	Liên hệ app thật
Model Class	Class chứa dữ liệu	Dùng Map cho nhanh	Route arguments
Enum Route	Enum định nghĩa route	Không cần thiết	Tránh sai tên
Null Safety	Dart chống null	Ignore cảnh báo	Crash runtime
Constructor	Hàm tạo màn hình	Nhét logic	Nhận arguments

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

🔍 PROMPT 1 – PHÂN TÍCH BẢN CHẤT NAVIGATION STACK

Prompt:
Phân tích vai trò của *Navigation Stack* trong Flutter.
So sánh cách quản lý stack của Navigator 1.0 với tư duy *page stack as state* trong Navigator 2.0.
Trong app có login/logout, cách quản lý stack nào giúp tránh lỗi back sai màn hình tốt hơn? Vì sao?

🎯 Trọng tâm:
stack ≠ danh sách màn hình đơn thuần
back behavior
auth flow

🔍 PROMPT 2 – ĐÁNH GIÁ NAMED ROUTES TRONG APP LỚN

Prompt:
Đánh giá ưu điểm và hạn chế của Named Routes khi số lượng màn hình tăng lên (20–30 screens).
Khi nào Named Routes trở thành gánh nặng?
Có những dấu hiệu nào cho thấy app cần chuyển sang routing nâng cao hơn?

🎯 Trọng tâm:
maintainability
scalability
dấu hiệu kiến trúc yếu

🔍 PROMPT 3 – ROUTE ARGUMENTS & QUẢN LÝ DỮ LIỆU ĐIỀU HƯỚNG

Prompt:
Phân tích các cách truyền dữ liệu giữa màn hình (constructor vs RouteSettings).
Trong app thương mại, dữ liệu nào nên truyền qua route, dữ liệu nào không?
Vì sao lạm dụng arguments có thể gây lỗi kiến trúc?

🎯 Trọng tâm:
data ownership
coupling giữa screens
sai lầm phổ biến

🔍 PROMPT 4 – NAVIGATION LÀ STATE HAY KHÔNG?

Prompt:
Phân tích quan điểm: *"Navigation cũng là một dạng state của ứng dụng"*.
Lấy ví dụ app có onboarding, login, role-based access để chứng minh hoặc phản biện quan điểm này.

🎯 Trọng tâm:
tư duy declarative
chuẩn bị cho state management
app flow

🔍 PROMPT 5 – KHI NÀO NAVIGATOR 1.0 KHÔNG ĐỦ?

Prompt:
Phân tích các tình huống thực tế mà Navigator 1.0 bắt đầu bộc lộ hạn chế.
Những biểu hiện nào trong code cho thấy app đang "vá navigation"?
Nếu không chuyển tư duy, hậu quả sẽ là gì khi app mở rộng?

🎯 Trọng tâm:
nhận diện vấn đề sớm
technical debt

PhucPT10@fpt.edu.vn

🔍 PROMPT 6 – DEEP LINK & KIẾN TRÚC APP

Prompt:
Phân tích ảnh hưởng của Deep Linking đến kiến trúc navigation của app Flutter.
Vì sao deep link buộc app phải có navigation rõ ràng ngay từ đầu?
Điều gì sẽ xảy ra nếu chỉ thiết kế navigation cho luồng nội bộ?

🎯 Trọng tâm:
external entry point
app thật vs app demo

🔍 PROMPT 7 – SO SÁNH NAVIGATION IMPERATIVE VS DECLARATIVE

Prompt:
So sánh tư duy *imperative navigation* và *declarative navigation*.
Mỗi tư duy phù hợp với loại app nào?
Vì sao Flutter và các framework hiện đại đang dịch chuyển sang declarative?

🎯 Trọng tâm:
xu hướng framework
kiến trúc dài hạn

🔍 PROMPT 8 – NAVIGATION & UX (GÓC NHÌN NGƯỜI DÙNG)

Prompt:
Phân tích mối quan hệ giữa navigation design và trải nghiệm người dùng (UX).
Một navigation thiết kế kém sẽ ảnh hưởng thế nào đến cảm nhận app, dù UI đẹp?

🎯 Trọng tâm:
user flow
cảm nhận sản phẩm
tư duy product

🔍 PROMPT 9 – TỔ CHỨC FILE & NAVIGATION LOGIC

Prompt:
Đánh giá tác động của việc tổ chức route, screen và navigation logic đến khả năng maintain app.
Vì sao navigation không nên viết rải rác trong UI?
Hậu quả khi team nhiều người cùng phát triển?

🎯 Trọng tâm:
teamwork
clean code nền tảng
mở rộng dự án

🔍 PROMPT 10 – TƯ DUY THIẾT KẾ NAVIGATION TRƯỚC KHI CODE

Prompt:
Phân tích vì sao navigation nên được thiết kế bằng sơ đồ user flow trước khi code.
So sánh hai cách tiếp cận:
(1) code trước – vá sau
(2) thiết kế luồng trước – code sau
Cách nào phù hợp với dự án chuyên nghiệp? Vì sao?

🎯 Trọng tâm:
tư duy kỹ sư phần mềm
khác biệt sinh viên vs developer chuyên nghiệp

🔍 PROMPT 1 – PHÂN TÍCH BẢN CHẤT NAVIGATION STACK

Prompt:
Phân tích vai trò của *Navigation Stack* trong Flutter.
So sánh cách quản lý stack của Navigator 1.0 với tư duy *page stack as state* trong Navigator 2.0.
Trong app có login/logout, cách quản lý stack nào giúp tránh lỗi back sai màn hình tốt hơn? Vì sao?

PhucPT10@fpt.edu.vn

🎯 Trọng tâm:
stack ≠ danh sách màn hình đơn thuần
back behavior
auth flow

🔍 PROMPT 2 – ĐÁNH GIÁ NAMED ROUTES TRONG APP LỚN

Prompt:
Đánh giá ưu điểm và hạn chế của Named Routes khi số lượng màn hình tăng lên (20–30 screens).
Khi nào Named Routes trở thành gánh nặng?
Có những dấu hiệu nào cho thấy app cần chuyển sang routing nâng cao hơn?

🎯 Trọng tâm:
maintainability
scalability
dấu hiệu kiến trúc yếu

🔍 PROMPT 3 – ROUTE ARGUMENTS & QUẢN LÝ DỮ LIỆU ĐIỀU HƯỚNG

Prompt:
Phân tích các cách truyền dữ liệu giữa màn hình (constructor vs RouteSettings).
Trong app thương mại, dữ liệu nào nên truyền qua route, dữ liệu nào không?
Vì sao lạm dụng arguments có thể gây lỗi kiến trúc?

🎯 Trọng tâm:
data ownership
coupling giữa screens
sai lầm phổ biến

🔍 PROMPT 4 – NAVIGATION LÀ STATE HAY KHÔNG?

Prompt:
Phân tích quan điểm: *"Navigation cũng là một dạng state của ứng dụng"*.
Lấy ví dụ app có onboarding, login, role-based access để chứng minh hoặc phản biện quan điểm này.

🎯 Trọng tâm:
tư duy declarative
chuẩn bị cho state management
app flow

🔍 PROMPT 5 – KHI NÀO NAVIGATOR 1.0 KHÔNG ĐỦ?

Prompt:
Phân tích các tình huống thực tế mà Navigator 1.0 bắt đầu bộc lộ hạn chế.
Những biểu hiện nào trong code cho thấy app đang "vá navigation"?
Nếu không chuyển tư duy, hậu quả sẽ là gì khi app mở rộng?

🎯 Trọng tâm:
nhận diện vấn đề sớm
technical debt

🔍 PROMPT 6 – DEEP LINK & KIẾN TRÚC APP

Prompt:
Phân tích ảnh hưởng của Deep Linking đến kiến trúc navigation của app Flutter.
Vì sao deep link buộc app phải có navigation rõ ràng ngay từ đầu?
Điều gì sẽ xảy ra nếu chỉ thiết kế navigation cho luồng nội bộ?

🎯 Trọng tâm:
external entry point
app thật vs app demo

PhucPT10@fpt.edu.vn

🔍 PROMPT 7 – SO SÁNH NAVIGATION IMPERATIVE VS DECLARATIVE

Prompt:
So sánh tư duy *imperative navigation* và *declarative navigation*.
Mỗi tư duy phù hợp với loại app nào?
Vì sao Flutter và các framework hiện đại đang dịch chuyển sang declarative?

🎯 Trọng tâm:
xu hướng framework
kiến trúc dài hạn

🔍 PROMPT 8 – NAVIGATION & UX (GÓC NHÌN NGƯỜI DÙNG)

Prompt:
Phân tích mối quan hệ giữa navigation design và trải nghiệm người dùng (UX).
Một navigation thiết kế kém sẽ ảnh hưởng thế nào đến cảm nhận app, dù UI đẹp?

🎯 Trọng tâm:
user flow
cảm nhận sản phẩm
tư duy product

🔍 PROMPT 9 – TỔ CHỨC FILE & NAVIGATION LOGIC

Prompt:
Đánh giá tác động của việc tổ chức route, screen và navigation logic đến khả năng maintain app.
Vì sao navigation không nên viết rải rác trong UI?
Hậu quả khi team nhiều người cùng phát triển?

🎯 Trọng tâm:
teamwork
clean code nền tảng
mở rộng dự án

🔍 PROMPT 10 – TƯ DUY THIẾT KẾ NAVIGATION TRƯỚC KHI CODE

Prompt:
Phân tích vì sao navigation nên được thiết kế bằng sơ đồ user flow trước khi code.
So sánh hai cách tiếp cận:
(1) code trước – vá sau
(2) thiết kế luồng trước – code sau
Cách nào phù hợp với dự án chuyên nghiệp? Vì sao?

🎯 Trọng tâm:
tư duy kỹ sư phần mềm
khác biệt sinh viên vs developer chuyên nghiệp

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

🧠 NHÓM A – NAVIGATION & KIẾN TRÚC APP LỚN

🔍 PROMPT 1 – NAVIGATION KHI APP CÓ 50+ MÀN HÌNH

Trong app Flutter có trên 50 màn hình, việc quản lý navigation bằng Named Routes thuần gặp những vấn đề gì?
Phân tích các rủi ro về maintainability, bug và onboarding developer mới.
Đề xuất cách tổ chức navigation phù hợp hơn cho app lớn.

🎯 Thực tế:
app enterprise
dự án nhiều năm

🔍 PROMPT 2 – NAVIGATION & TECHNICAL DEBT

Phân tích cách navigation được code vội vàng ở giai đoạn MVP có thể trở thành technical debt như thế nào khi app mở rộng.
Những dấu hiệu nào cho thấy navigation đang "nợ kỹ thuật"?

🎯 Thực tế:
startup
refactor đau đớn

🔍 PROMPT 3 – NAVIGATION LOGIC Ở UI LÀ SAI Ở ĐÂU?

Phân tích hậu quả khi logic navigation được viết trực tiếp trong UI widget.
Điều gì xảy ra khi cần thay đổi user flow hoặc thêm deep link?

🎯 Thực tế:
khó test
khó refactor

🧠 NHÓM B – USER FLOW, AUTH & STATE

🔍 PROMPT 4 – AUTH FLOW & NAVIGATION

Phân tích cách navigation thay đổi khi app có login, logout, token hết hạn.
Vì sao không thể chỉ dùng push/pop cho auth flow trong app thật?

🎯 Thực tế:
app thương mại
bảo mật

🔍 PROMPT 5 – ROLE-BASED NAVIGATION

Trong app có nhiều vai trò (admin, user, staff), navigation cần được thiết kế như thế nào?
Phân tích rủi ro khi không tách rõ luồng điều hướng theo role.

🎯 Thực tế:
app nội bộ doanh nghiệp

🔍 PROMPT 6 – NAVIGATION LÀ STATE: ỨNG DỤNG THỰC TẾ

Phân tích cách xem navigation như một phần của state giúp app ổn định hơn trong các tình huống phức tạp (logout, refresh, deep link).

🎯 Thực tế:
app chạy lâu
ít bug back stack

PhucPT10@fpt.edu.vn

🧠 NHÓM C – DEEP LINK & EXTERNAL ENTRY

🔍 PROMPT 7 – DEEP LINK TRONG APP THƯƠNG MẠI

Phân tích vai trò của deep link trong app e-commerce (quảng cáo, chia sẻ sản phẩm, notification).
Vì sao deep link ảnh hưởng trực tiếp đến kiến trúc navigation?

🎯 Thực tế:
marketing
tăng conversion

🔍 PROMPT 8 – XỬ LÝ DEEP LINK KHI APP CHƯA LOGIN

Khi user mở app từ deep link nhưng chưa login, app nên xử lý navigation thế nào?
Phân tích các phương án và rủi ro UX.

🎯 Thực tế:
onboarding
auth flow

🔍 PROMPT 9 – DEEP LINK & BACK STACK

Phân tích cách deep link ảnh hưởng đến back stack.
Nếu mở thẳng màn hình chi tiết, back nên quay về đâu?

🎯 Thực tế:
UX
Android behavior

🧠 NHÓM D – TEAMWORK & MAINTAINABILITY

🔍 PROMPT 10 – NAVIGATION TRONG TEAM NHIỀU DEV

Phân tích các vấn đề navigation phát sinh khi nhiều developer cùng phát triển app Flutter.
Navigation cần được chuẩn hóa thế nào để tránh conflict và bug?

🎯 Thực tế:
teamwork
code review

🔍 PROMPT 11 – TESTING NAVIGATION

Vì sao navigation khó test hơn UI thường?
Phân tích các chiến lược giúp test navigation dễ hơn trong dự án lớn.

🎯 Thực tế:
QA
CI/CD

🔍 PROMPT 12 – DEBUG NAVIGATION TRONG PRODUCTION

Khi app production gặp lỗi navigation (back sai, mở sai màn), việc debug khó ở điểm nào?
Navigation design tốt giúp debug dễ hơn ra sao?

🎯 Thực tế:
log
crash report

🧠 NHÓM E – SO SÁNH & ĐỊNH HƯỚNG DÀI HẠN

🔍 PROMPT 13 – KHI NÀO NÊN CHUYỂN SANG ROUTER API / GOROUTER?

Phân tích các tiêu chí thực tế để quyết định chuyển từ Navigator 1.0 sang Router API hoặc GoRouter.
Vì sao chuyển quá sớm cũng là rủi ro?

🎯 Thực tế:
over-engineering

PhucPT10@fpt.edu.vn

🔍 PROMPT 14 – NAVIGATION & CROSS-PLATFORM

Phân tích sự khác biệt về navigation giữa mobile, web và desktop trong Flutter.
Vì sao navigation cần được thiết kế linh hoạt ngay từ đầu?

🎯 Thực tế:
Flutter đa nền tảng

🔍 PROMPT 15 – NAVIGATION LÀ TRẢI NGHIỆM SẢN PHẨM

Phân tích vì sao navigation không chỉ là vấn đề kỹ thuật mà còn là vấn đề sản phẩm.
Một navigation thiết kế kém ảnh hưởng thế nào đến retention và đánh giá app?

🎯 Thực tế:
product thinking
business impact

PhucPT10@fpt.edu.vn